GRUPO PENTANDRA

// TECH CHALLENGE 360

Reto ShopCart

Enunciado del proyecto. Construye una plataforma web de ventas de nivel productivo, con base de datos, API y frontend, y defiéndela frente a nuestro equipo técnico.

INICIO	FASE 1	FASE 2	NIVEL
Lun 26 oct 2026	Lun 2 nov · 23:59	Lun 9 nov · 23:59	Middle / Senior

// 01

Información general

El participante debe construir en 15 días naturales una plataforma web de ventas que incluya base de datos, API y frontend.

El sistema deberá permitir administrar productos, categorías, inventario y promociones, así como realizar ventas mediante un carrito de compras.

El objetivo es evaluar su capacidad para diseñar, implementar y defender una solución de nivel productivo de forma autónoma.

ASPECTO	DETALLE
Nombre del proyecto	ShopCart
Duración	15 días naturales, del lunes 26 de octubre al lunes 9 de noviembre de 2026
Nivel evaluado	Desarrollador Full Stack Middle o Senior
Modalidad	Individual
Base de datos	Microsoft SQL Server 2019 o superior
Backend	Node.js 24 LTS o superior con TypeScript (Express)
Frontend	React 18 o superior con TypeScript
Entrega final	Repositorio Git, ambiente ejecutable y demo con defensa técnica
Acceso a datos	Exclusivamente mediante Stored Procedures

El participante puede usar las librerías y herramientas que considere convenientes, siempre que respete el stack indicado y justifique sus decisiones en el README.

Fases y fechas de entrega

El reto inicia el lunes 26 de octubre de 2026 y se entrega en dos fases, cada una con cierre en lunes. Cada fase se evalúa con el último commit anterior a su fecha límite.

FASE 1

Base de datos y backend

Fecha límite: lunes 2 de noviembre de 2026, 23:59 horas

FASE 2

Frontend y proyecto completo

Fecha límite: lunes 9 de noviembre de 2026, 23:59 horas

// 02

Contexto y problema a resolver

Una cadena de tiendas vende hoy solo en sus puntos físicos y quiere lanzar su tienda en línea. Necesita una plataforma propia donde los clientes compren desde el navegador y el equipo interno administre catálogo, inventario, promociones y pedidos.

El flujo principal es el de cualquier comercio electrónico: el cliente navega el catálogo, agrega productos al carrito, aplica cupones, confirma dirección y envío, paga y da seguimiento a su pedido. Del lado interno, el equipo prepara y despacha los pedidos, controla existencias y analiza las ventas.

El negocio tiene preocupaciones concretas que la plataforma debe resolver:

- No vender productos sin existencia, aunque varios clientes compren la última unidad al mismo tiempo.
- Respetar el precio y el descuento que el cliente vio al confirmar la compra, aunque cambien después.
- No cobrar ni crear pedidos duplicados si el cliente presiona dos veces el botón de pagar o falla la conexión.
- Conocer ventas, productos más vendidos y carritos abandonados sin esperar reportes manuales.
- Tener trazabilidad completa de quién modificó precios, existencias y pedidos.
- Contar con trazabilidad clara de entradas, salidas y ajustes de inventario.

MONEDA E IMPUESTOS

Los montos se manejan en quetzales y los precios publicados incluyen IVA del 12 %.

// 03

Glosario

TÉRMINO	DEFINICIÓN
Producto	Artículo del catálogo con nombre, descripción, categoría e imágenes
Variante	Combinación vendible de un producto (por ejemplo talla y color); tiene su propio SKU, precio y existencia
SKU	Código único que identifica una variante
Carrito de invitado	Carrito de un visitante sin sesión, identificado por un token
Reserva de existencia	Apartado temporal de unidades durante el checkout, que se libera si el pago no se completa
Checkout	Proceso de confirmar dirección, envío, cupón y pago para convertir el carrito en pedido
Pedido	Compra confirmada con su detalle de precios congelado al momento de la compra
Cupón	Código de descuento con reglas de vigencia, monto mínimo y límite de usos
Kardex	Historial de entradas, salidas y ajustes de existencia por variante
Carrito abandonado	Carrito con productos sin actividad durante un periodo configurable
Ticket promedio	Monto total vendido entre número de pedidos pagados en un periodo
Idempotencia	Propiedad de una operación que produce el mismo resultado aunque se repita

// 04

Requerimientos funcionales

La solución debe implementar los 9 módulos siguientes. Cada módulo indica sus reglas de negocio y los criterios de aceptación que se verificarán en la evaluación.

RF-01 Autenticación y control de acceso

El sistema maneja tres roles: Cliente, Operador y Administrador.

- Registro de clientes con correo y contraseña; contraseñas almacenadas con hash seguro.
- Autenticación con access token JWT de corta duración y refresh token con rotación.
- El Cliente compra y consulta solo su propia información: carrito, pedidos y direcciones.
- El Operador gestiona pedidos e inventario. El Administrador gestiona además catálogo, precios, cupones, usuarios y reportes.
- Un usuario puede desactivarse sin borrarse.

CRITERIOS DE ACEPTACIÓN

- Las restricciones se validan en el backend. Cambiar el identificador de un pedido, carrito o dirección en la URL o en el body no debe exponer ni modificar datos de otro cliente.
- Un token expirado devuelve 401; un acceso a un recurso no permitido devuelve 403.

RF-02 Catálogo de productos

- CRUD de categorías.
- CRUD de productos con variantes. Cada variante tiene SKU único, atributos (por ejemplo talla y color), precio y existencia propios.
- Búsqueda por texto y filtros por categoría, rango de precio y disponibilidad, con ordenamiento y paginación.
- Los productos con ventas asociadas no se eliminan físicamente; se desactivan y dejan de mostrarse en la tienda.

CRITERIOS DE ACEPTACIÓN

- Un cambio de precio no altera el monto de pedidos ya realizados.

RF-03 Inventario

- Cada movimiento de existencia (entrada por compra, salida por venta, ajuste, devolución) queda registrado en el kardex con usuario, fecha, motivo y cantidad.
- La existencia disponible es la existencia física menos las unidades reservadas.

CRITERIOS DE ACEPTACIÓN

- La existencia actual coincide con la suma de movimientos del kardex.
- Un ajuste manual exige motivo y no puede dejar la existencia en negativo.

RF-04 Carrito de compras

- Agregar, modificar cantidad y eliminar productos del carrito.
- El carrito muestra subtotal, descuentos y total.

CRITERIOS DE ACEPTACIÓN

- No se puede agregar más cantidad que la existencia disponible.
- El total del carrito no presenta errores de redondeo.

RF-05 Cupones y promociones

- CRUD de cupones de tipo porcentaje o monto fijo.
- Cada cupón tiene vigencia, monto mínimo de compra, límite total de usos y límite de usos por cliente.
- Un cupón puede aplicar a todo el carrito o solo a una categoría.
- Solo se permite un cupón por pedido.

CRITERIOS DE ACEPTACIÓN

- Un cupón vencido, agotado o que no cumple el mínimo se rechaza con un mensaje específico.
- El límite total de usos se respeta aunque varios clientes usen el cupón al mismo tiempo.
- Un descuento nunca deja el total del pedido por debajo de cero.

RF-06 Checkout y pago

- El cliente selecciona o registra una dirección de entrega.
- Al iniciar el pago, el sistema reserva la existencia de cada variante del carrito durante 15 minutos.

CRITERIOS DE ACEPTACIÓN

- Si dos clientes intentan comprar la última unidad al mismo tiempo, solo uno lo logra y el otro recibe un mensaje claro. Nunca hay sobreventa.
- Repetir la misma solicitud de pago (doble clic o reintento por falla de red) no genera dos pedidos ni dos cobros.
- La creación del pedido es atómica: pedido, detalle, movimientos de inventario y uso de cupón se guardan juntos o no se guarda nada.

RF-07 Gestión de pedidos y devoluciones

- El participante define el ciclo de vida del pedido (estados y transiciones permitidas), desde su creación hasta la entrega, incluyendo cancelación y devolución, y lo documenta en el README con un diagrama de estados.
- El Operador prepara, despacha y avanza los pedidos de estado; el Cliente consulta el estado y el historial de sus propios pedidos.
- Un pedido puede cancelarse mientras no haya sido despachado.
- Las devoluciones pueden ser totales o parciales por línea de pedido.

CRITERIOS DE ACEPTACIÓN

- Una transición de estado no permitida se rechaza en el backend y en la base de datos.
- Cancelar o devolver reintegra la existencia mediante movimientos en el kardex y se refleja en los reportes.
- Una reserva vencida libera la existencia sin intervención manual.
- No se puede devolver más cantidad de la vendida en cada línea del pedido.
- Cada cambio de estado registra usuario, fecha y hora, estado anterior y estado nuevo.

RF-08 Dashboard y reportes

- Ventas por día, semana y mes, en quetzales y en número de pedidos.
- Ventas por categoría y los 10 productos más vendidos de un periodo.
- Listado de pedidos con filtros por estado, cliente y rango de fechas.

CRITERIOS DE ACEPTACIÓN

- Los listados usan paginación y filtros del lado del servidor.
- Los reportes solo consideran pedidos completados, excluyendo cancelados y descontando devoluciones.

RF-09 Bitácora

- Toda creación, modificación o desactivación en productos, precios, existencias, cupones y estados de pedido queda registrada.
- Cada registro guarda usuario, fecha y hora, entidad, acción, valor anterior y valor nuevo.
- La bitácora es consultable por los Administradores, con filtro por módulo.

CRITERIOS DE ACEPTACIÓN

- La bitácora no puede modificarse ni eliminarse.

// 05

Requerimientos técnicos obligatorios

Estos requisitos son obligatorios; su ausencia resta puntos aunque cumpla con la funcionalidad requerida.

Base de datos (SQL Server)

- Modelo relacional normalizado, con llaves primarias, foráneas, restricciones UNIQUE, NOT NULL y CHECK donde aplique.
- Montos y volúmenes con tipo DECIMAL o INTEGER con precisión definida. No se permite FLOAT ni REAL para estos datos.
- Scripts SQL versionados y numerados que crean la base de datos, sus objetos y los datos semilla de forma reproducible.
- Todo el acceso a datos (creación, edición y lectura) se hace mediante stored procedures.

Backend (Node.js + TypeScript)

- Express con TypeScript en modo strict.
- Arquitectura por capas: rutas o controladores, servicios con la lógica de negocio y acceso a datos.
- Implementar validaciones en el API para garantizar que los datos recibidos cumplan con el esquema esperado.
- Prevención de SQL injection: los SPs se invocan con parámetros tipados. No se permite concatenar valores en cadenas SQL, ni en el backend ni dentro de los SPs; si un SP necesitara utilizar SQL dinámico, debe usar `sp_executesql` con parámetros.
- Manejo centralizado de errores con respuestas consistentes y códigos HTTP correctos.
- Configuración por variables de entorno, con un archivo `.env.example`.

Frontend (React + TypeScript)

- Rutas protegidas y menús según el rol del usuario.
- Formularios con validación y mensajes de error claros.
- Tablas con paginación, ordenamiento y filtros del lado del servidor.
- Gráficas en el dashboard.
- Estados visibles de carga, vacío y error en todas las vistas.
- Renovación transparente del token con el refresh token.
- Interfaz responsive.

Calidad y entorno

- README con instrucciones de instalación, decisiones técnicas, diagrama entidad-relación y credenciales de prueba.
- Historial de commits legible y progresivo; no se acepta un único commit al final.

Estándares de base de datos

Todos los objetos de SQL Server deben seguir el estándar de nomenclatura de la empresa. Los nombres usan la primera letra de cada palabra en mayúscula, igual que el prefijo del tipo de objeto.

OBJETO	FORMATO	EJEMPLOS PARA ESTE PROYECTO
Stored procedure	`SP_Modulo_Proceso`	`SP_Pedidos_CrearPedido` `SP_Inventario_ReservarExistencia`
Vista	`VW_Modulo_Proceso`	`VW_Dashboard_VentasPorCategoria` `VW_Pedidos_Historial`
Función tipo tabla	`FU_Modulo_Proceso`	`FU_Cupones_CalculoDescuento` `FU_Catalogo_PrecioVigente`
Llave primaria autoincrementable	ID en mayúsculas seguido del nombre con la primera letra en mayúscula	`IDProducto` `IDCarrito` `IDPedido` `IDCupon`

Todos los objetos se crean con scripts SQL escritos por el participante; no se aceptan objetos generados por un ORM.

// 06

Requerimientos no funcionales

CATEGORÍA	REQUISITO
Rendimiento	Los listados y reportes principales responden en menos de 1 segundo con los datos semilla
Seguridad	Sin credenciales en el repositorio; CORS configurado; cabeceras de seguridad básicas; contraseñas con hash
Integridad	Ninguna operación deja datos a medias ante un error; las reglas críticas también se protegen en la base de datos
Concurrencia	El checkout, el uso de cupones y los movimientos de inventario soportan acceso simultáneo sin sobreventa ni inconsistencias
Mantenibilidad	Código tipado, nombres claros, sin lógica de negocio en controladores ni en componentes visuales
Portabilidad	El proyecto corre en una máquina limpia siguiendo solo el README

DECISIONES ABIERTAS

El enunciado deja abiertas algunas decisiones a propósito. El participante debe identificarlas, tomar una postura razonable y documentarla en el README. La claridad de esas decisiones forma parte de la evaluación.

// 07

Entregables

Entregables finales, según la fecha límite de cada fase (ver Fases y fechas de entrega):

- [ ] Repositorio Git con acceso para el evaluador.
- [ ] README completo según la sección 5.
- [ ] Diagrama entidad-relación.

// 08

Criterios de evaluación

La calificación es sobre 100 puntos y se aprueba con 70. Se valora más la calidad y la corrección de lo entregado que la cantidad de pantallas.

CRITERIO	QUÉ SE EVALÚA	PUNTOS
Modelo de datos e integridad	Normalización, tipos de datos, constraints, índices, transacciones, migraciones y apego al estándar de base de datos	20
Lógica de negocio	Cumplimiento de los criterios de aceptación de RF-01 a RF-09, incluidos los casos borde	20
Defensa técnica	Claridad al explicar decisiones, trade-offs y mejoras posibles	20
Backend	Arquitectura por capas, validación, manejo de errores, seguridad, documentación de la API y apego al estándar de código	15
Frontend	Usabilidad, manejo de estado, control por rol, estados de carga y error, uso de componentes reutilizables y apego al estándar de código	15
Entorno y documentación	README, semillas, decisiones documentadas e historial de commits	10
Total		100

En la defensa técnica el evaluador puede pedir modificaciones pequeñas en vivo, revisar cualquier parte del código o plantear escenarios nuevos sobre las reglas de negocio.

PENALIZACIÓN DIRECTA

- Funcionalidad presentada que no corre en el ambiente entregado.
- Un cliente puede ver o modificar pedidos, carritos o direcciones de otro cliente.
- Código copiado sin entendimiento demostrable en la defensa.
- Uso de un ORM, consultas SQL en el código del backend o cualquier punto vulnerable a SQL injection.
- Problemas de rendimiento.

// 09

Extras opcionales

Los extras suman hasta 10 puntos adicionales, solo si los requerimientos obligatorios están completos y funcionando. Un extra no compensa un requerimiento obligatorio faltante.

- Actualización en tiempo real del dashboard con WebSockets o Server-Sent Events.
- Exportación de reportes a Excel o PDF.
- Caché de reportes con invalidación correcta.
- Rate limiting en los endpoints de autenticación.

// 10

Reglas generales

- El trabajo es individual. Se permite consultar documentación, foros y herramientas de IA, pero el participante debe poder explicar y modificar cualquier parte de su código.
- Las dudas sobre el enunciado se envían por escrito al evaluador. Las respuestas se comparten con todos los participantes.
- Donde el enunciado no especifique algo, el participante decide y documenta su supuesto.
- No se aceptan commits posteriores a la fecha y hora de entrega.